

Project

Skilyn Leon

Table of contents

The network (G)	2
Table of G (ranked)	2
Analysis	5
Appendix	6
Adjacency matrix of G	6

The network (G)

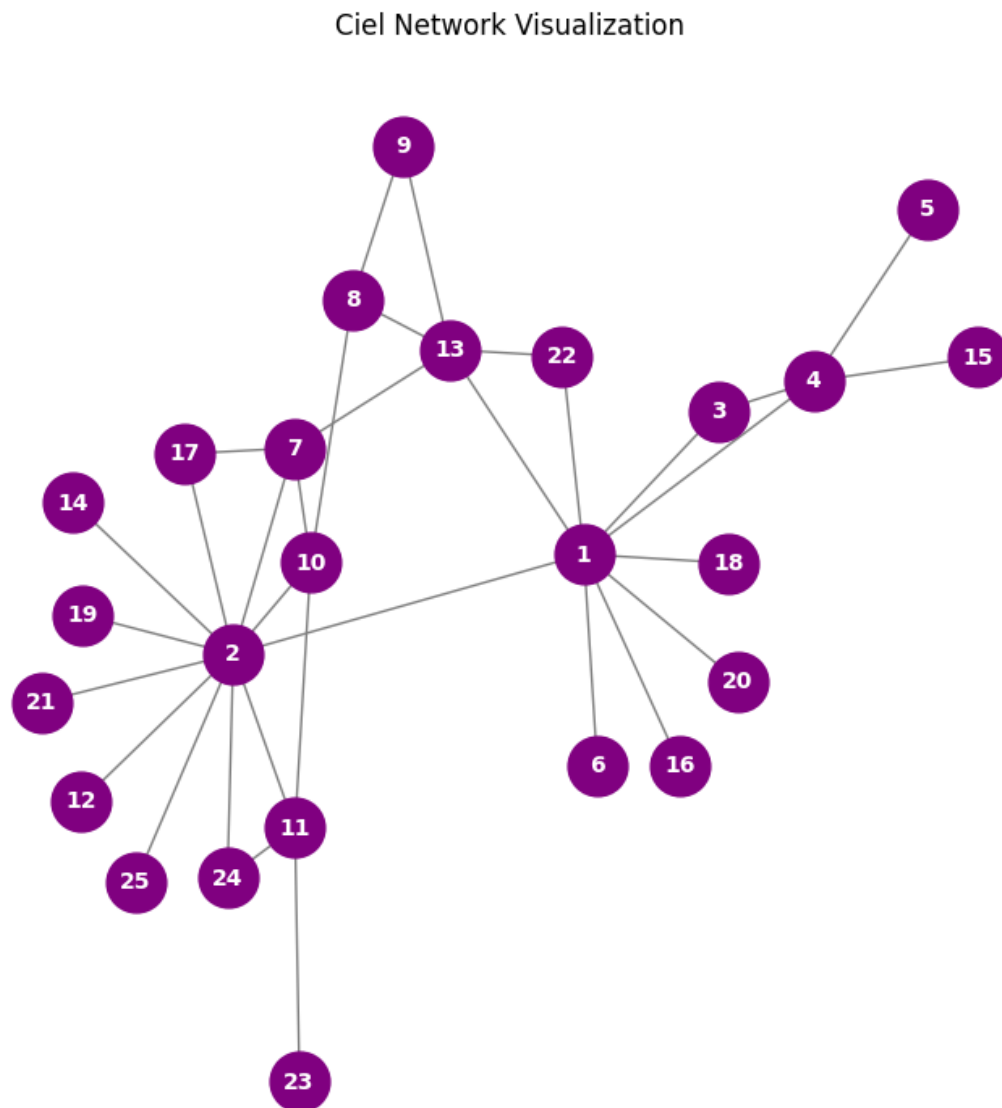
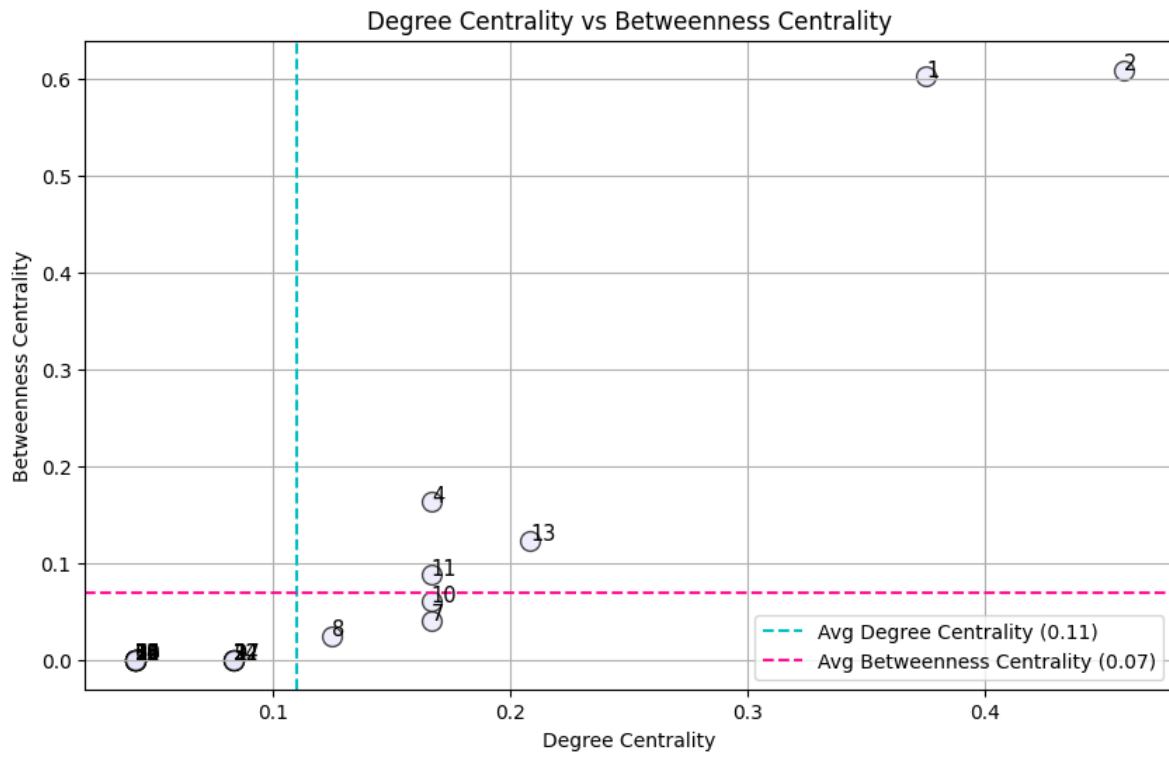
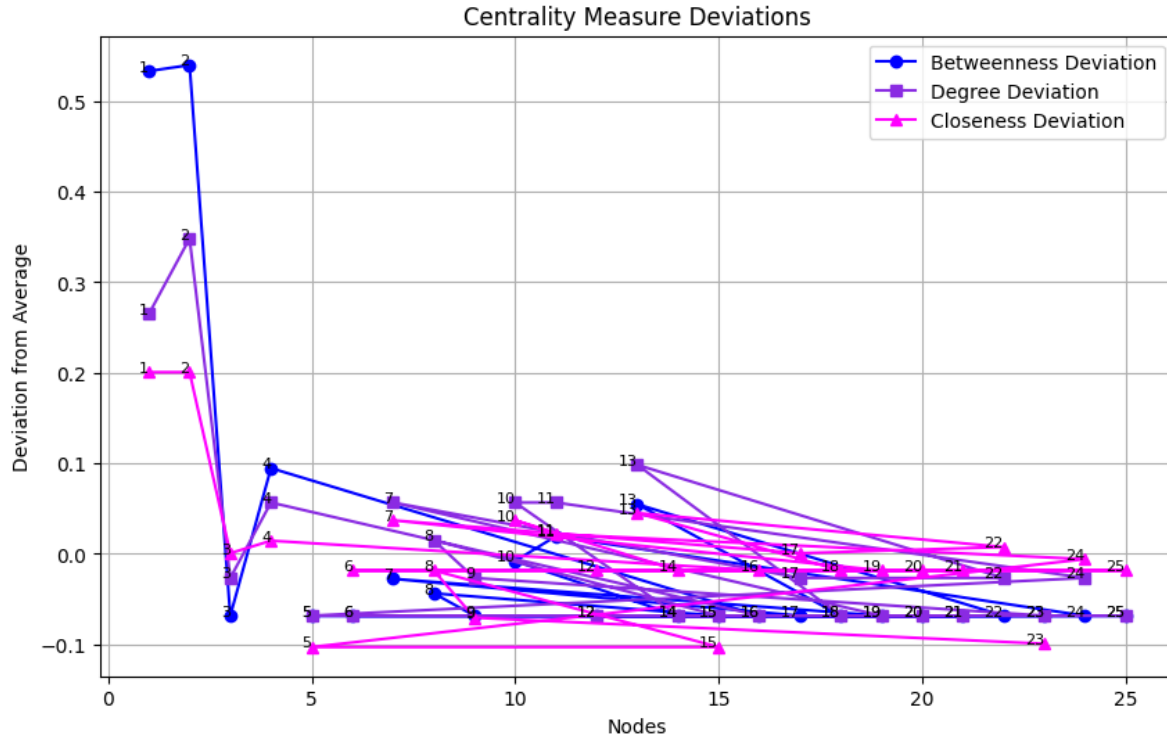


Figure 1: Ciel network plot

Table of G (ranked)

Node	Clustering Coefficient	Degree Centrality (Rank)	Betweenness Centrality	Closeness Centrality
1	0.05 (6)	3/8 (3)	0.6 (2)	0.6 (1)
2	0.07 (5)	0.46 (1)	0.6 (1)	0.6 (1)
3	1 (1)	0.08 (6)	0 (9)	0.4 (7)
4	1/6 (4)	1/6 (4)	0.16 (4)	0.41 (8)
5	0 (7)	0.04 (7)	0 (9)	0.3 (12)
6	0 (7)	0.04 (7)	0 (9)	0.38 (9)
7	1/3 (2)	1/6 (4)	0.04 (7)	0.44 (3)
8	1/3 (2)	1/8 (5)	0.2 (8)	0.38 (9)
9	1 (1)	0.08 (6)	0 (9)	0.33 (10)
10	1/3 (2)	1/6 (4)	0.06 (6)	0.44 (3)
11	1/3 (2)	1/6 (4)	0.09 (5)	0.42 (4)
12	0 (7)	0.04 (7)	0 (9)	0.38 (9)
13	0.2 (3)	0.2 (2)	0.12 (3)	4/9 (2)
14	0 (7)	0.04 (7)	0 (9)	0.38 (9)
15	0 (7)	0.04 (7)	0 (9)	0.3 (11)
16	0 (7)	0.04 (7)	0 (9)	0.38 (9)
17	1 (1)	0.08 (6)	0 (9)	0.4 (7)
18	0 (7)	0.04 (7)	0 (9)	0.38 (9)
19	0 (7)	0.04 (7)	0 (9)	0.38 (9)
20	0 (7)	0.04 (7)	0 (9)	0.38 (9)
21	0 (7)	0.04 (7)	0 (9)	0.38 (9)
22	1 (1)	0.08 (6)	0 (9)	0.41 (5)
23	0 (7)	0.04 (7)	0 (9)	0.3 (12)
24	1 (1)	0.08 (6)	0 (9)	0.4 (6)
25	0 (7)	0.04 (7)	0 (9)	0.38 (9)





Analysis

The calculated density of the network is 0.11, which is quite low. This suggests that the network is sparse, meaning most nodes do not have direct connections with many others. Instead, the network relies on key intermediaries for communication and coordination. Degree Centrality identifies the most connected nodes, the ones with the highest number of direct connections. Betweenness Centrality reveals nodes that act as bridges or intermediaries. Closeness Centrality highlights nodes that can quickly reach others in the network. Clustering Coefficient measures how connected a node's neighbors are to each other, indicating local connectivity. With this in mind, node 2 is likely the ring leader with rank 1 in Degree, Betweenness, and Closeness. Node 1 is likely the second in command (lieutenant) since its rank close second to node 2. Node 7 commuantes with node 2 the most other than node 1. Drug Mules are nodes with medium-high connectivity but low clustering (10, 7, 24). Complementary Segment are nodes 8, 9, 16, 21, 23 with strong internal cliques but not as connected to the larger network. In order to dismantle the network removing nodes 1, 2, 13, 7 would disrupt the major connections.

Appendix

```
#run these first
import numpy as np
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
```

```
G = nx.Graph()
graph_edges = [
    (1, 2), (1, 3), (1, 4), (1,16), (1,6), (1,20), (1,18), (1,13), (1,22),
    (2, 17), (2, 7), (2, 19), (2,21), (2,12), (2,25), (2,14), (2,10), (2,11), (2,24),
    (3, 4),
    (4,5), (4,15),
    (7, 17), (7, 13), (7,10),
    (8,9), (8, 10), (8,13),
    (9, 13),
    (10, 11),
    (11, 24), (11, 23),
    (13,22)
]
G.add_edges_from(graph_edges)
```

Adjacency matrix of G

```
#creating adjancency matrix
adjacency_matrix = nx.to_numpy_array(G, nodelist=sorted(G.nodes()), dtype=int)
print("Symmetric Adjacency Matrix:")
print(adjacency_matrix)
```

```
Symmetric Adjacency Matrix:
[[0 1 1 1 0 1 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0]
 [1 0 0 0 0 0 1 0 0 1 1 1 0 1 0 0 1 0 1 0 1 0 1]
 [1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [1 0 1 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
 [0 1 0 0 0 0 0 0 0 1 0 0 1 0 0 0 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0]
```

```

[0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

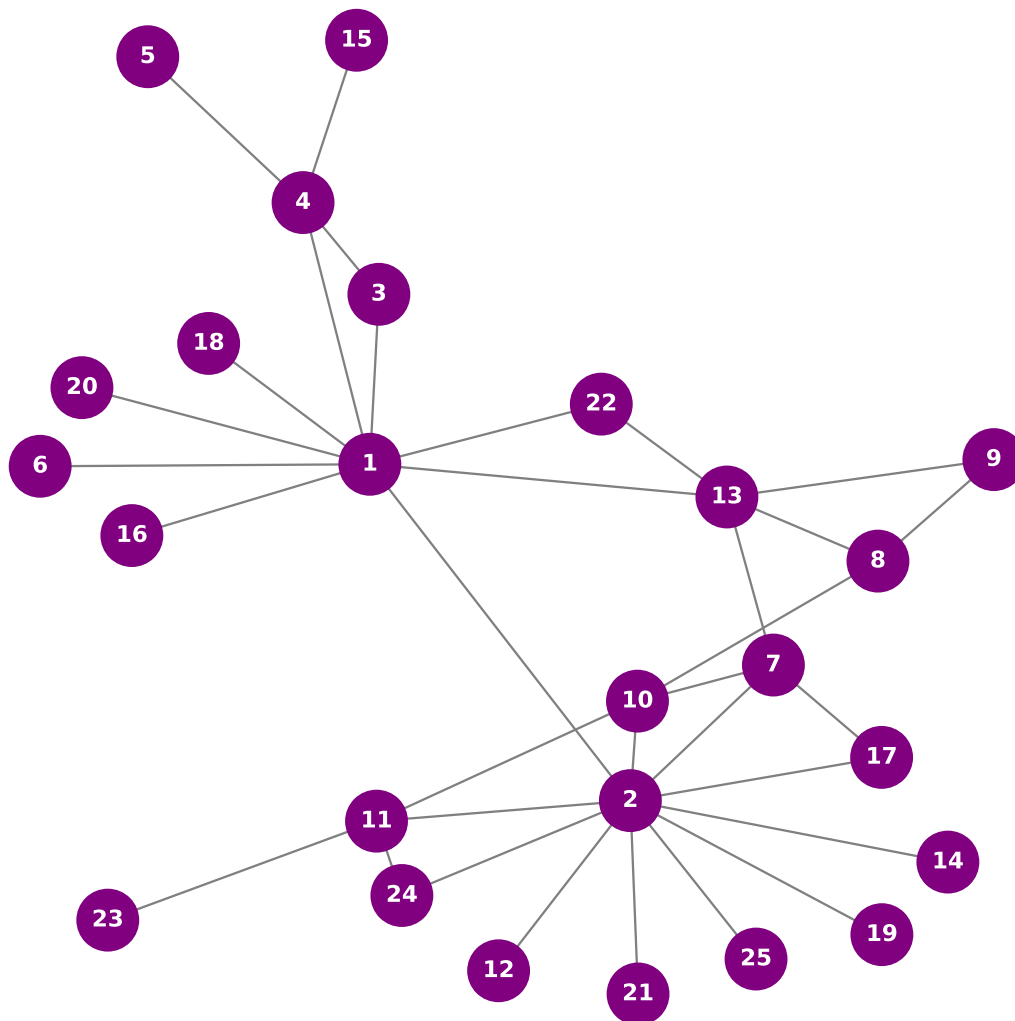
```

```

#recreating ciel network
plt.figure(figsize = (7, 7))
nx.draw(G, with_labels=True, node_color='purple', node_size=700, font_size=10, font_color='w')
plt.title("Ciel Network Visualization")
plt.show()

```

Ciel Network Visualization



```
#Calculating the degree centrality, betweenness centrality, clustering coefficient, closeness
degree_centrality = nx.degree_centrality(G)
betweenness_centrality = nx.betweenness_centrality(G)
closeness_centrality = nx.closeness_centrality(G)
clustering_coefficient = nx.clustering(G)
density = nx.density(G)
avg_clustering = nx.average_clustering(G)
#making a data frame to show the values
```

```

metrics_df = pd.DataFrame({
    'Node': sorted(G.nodes()),
    'Degree Centrality': [degree centrality[node] for node in sorted(G.nodes())],
    'Betweenness Centrality': [betweenness centrality[node] for node in sorted(G.nodes())],
    'Closeness Centrality': [closeness centrality[node] for node in sorted(G.nodes())],
    'Clustering Coefficient': [clustering_coefficient[node] for node in sorted(G.nodes())],
    'avg_clustering': avg_clustering,
    'Density': density
})
print(metrics_df)

```

	Node	Degree Centrality	Betweenness Centrality	Closeness Centrality \
0	1	0.375000	0.601751	0.600000
1	2	0.458333	0.608394	0.600000
2	3	0.083333	0.000000	0.400000
3	4	0.166667	0.163043	0.413793
4	5	0.041667	0.000000	0.296296
5	6	0.041667	0.000000	0.380952
6	7	0.166667	0.040761	0.436364
7	8	0.125000	0.024758	0.380952
8	9	0.083333	0.000000	0.328767
9	10	0.166667	0.060386	0.436364
10	11	0.166667	0.087862	0.421053
11	12	0.041667	0.000000	0.380952
12	13	0.208333	0.123188	0.444444
13	14	0.041667	0.000000	0.380952
14	15	0.041667	0.000000	0.296296
15	16	0.041667	0.000000	0.380952
16	17	0.083333	0.000000	0.400000
17	18	0.041667	0.000000	0.380952
18	19	0.041667	0.000000	0.380952
19	20	0.041667	0.000000	0.380952
20	21	0.041667	0.000000	0.380952
21	22	0.083333	0.000000	0.406780
22	23	0.041667	0.000000	0.300000
23	24	0.083333	0.000000	0.393443
24	25	0.041667	0.000000	0.380952

	Clustering Coefficient	avg_clustering	Density
0	0.055556	0.273131	0.11
1	0.072727	0.273131	0.11
2	1.000000	0.273131	0.11

3	0.166667	0.273131	0.11
4	0.000000	0.273131	0.11
5	0.000000	0.273131	0.11
6	0.333333	0.273131	0.11
7	0.333333	0.273131	0.11
8	1.000000	0.273131	0.11
9	0.333333	0.273131	0.11
10	0.333333	0.273131	0.11
11	0.000000	0.273131	0.11
12	0.200000	0.273131	0.11
13	0.000000	0.273131	0.11
14	0.000000	0.273131	0.11
15	0.000000	0.273131	0.11
16	1.000000	0.273131	0.11
17	0.000000	0.273131	0.11
18	0.000000	0.273131	0.11
19	0.000000	0.273131	0.11
20	0.000000	0.273131	0.11
21	1.000000	0.273131	0.11
22	0.000000	0.273131	0.11
23	1.000000	0.273131	0.11
24	0.000000	0.273131	0.11

```
#creating a plot for show Degree centrality vs Betweenness Centrality
plt.figure(figsize=(10, 6))
plt.scatter(degree_centrality.values(), betweenness_centrality.values(), color='lavender', edgecolor='black')

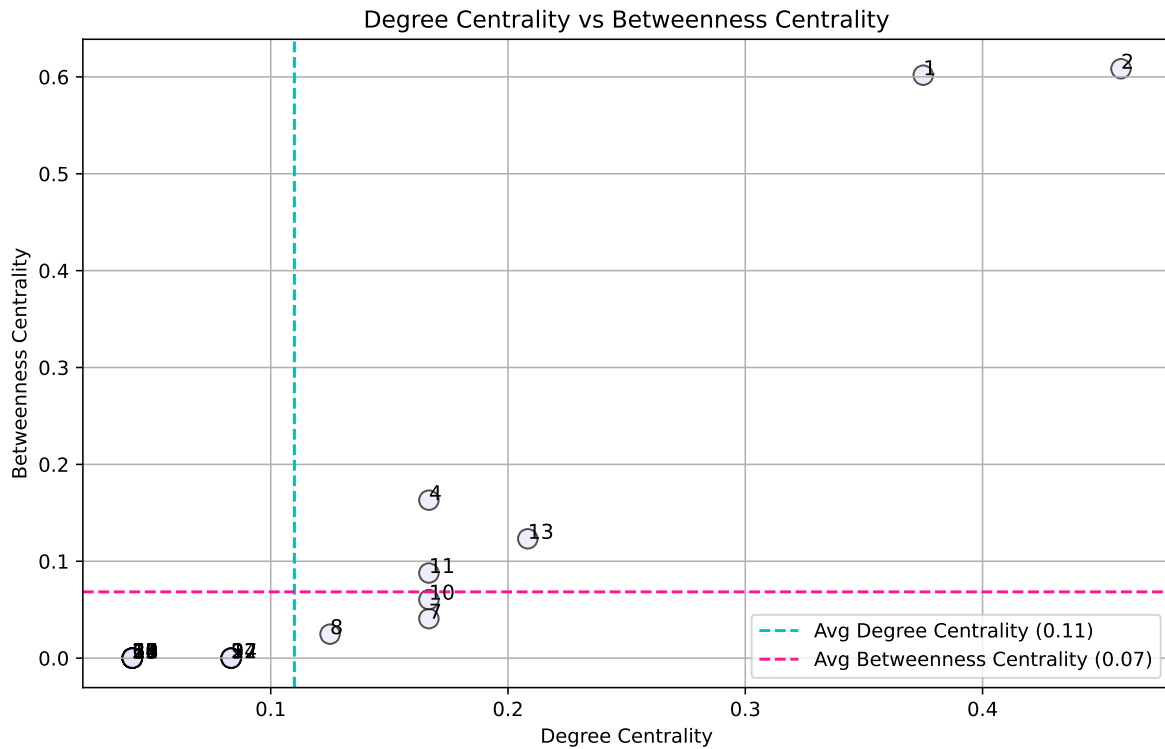
for node in G.nodes():
    plt.text(degree_centrality[node], betweenness_centrality[node], str(node), fontsize=10.5)

avg_degree = np.mean(list(degree_centrality.values()))
avg_betweenness = np.mean(list(betweenness_centrality.values()))

plt.axvline(avg_degree, color='c', linestyle='dashed', label=f'Avg Degree Centrality ({avg_degree})')
plt.axhline(avg_betweenness, color='deeppink', linestyle='dashed', label=f'Avg Betweenness Centrality ({avg_betweenness})')

plt.xlabel("Degree Centrality")
plt.ylabel("Betweenness Centrality")
plt.title("Degree Centrality vs Betweenness Centrality")
plt.legend()
plt.grid(True)
```

```
plt.show()
```



```
#Finding the vaules
avg_closeness = np.mean(list(closeness centrality.values()))
betweenness_deviation = {node: value - avg_betweenness for node, value in betweenness_centra
degree_deviation = {node: value - avg_degree for node, value in degree Centrality.items()}
closeness_deviation = {node: value - avg_closeness for node, value in closeness Centrality.items()}
#creating a data frame to show values
dataframe = pd.DataFrame({
    'Node': sorted(G.nodes()),
    'Betweenness Deviation': [betweenness_deviation[node] for node in sorted(G.nodes())],
    'Degree Deviation': [degree_deviation[node] for node in sorted(G.nodes())],
    'Closeness Deviation': [closeness_deviation[node] for node in sorted(G.nodes())]
})
print(dataframe)

plt.figure(figsize=(10, 6))

# Plot Betweenness Deviation
plt.plot(list(betweenness_deviation.keys()), list(betweenness_deviation.values()), label='Be
```

```

for node, value in betweenness_deviation.items():
    plt.text(node, value, str(node), fontsize=8, ha='right')

# Plot Degree Deviation
plt.plot(list(degree_deviation.keys()), list(degree_deviation.values()), label='Degree Devia
for node, value in degree_deviation.items():
    plt.text(node, value, str(node), fontsize=8, ha='right')

# Plot Closeness Deviation
plt.plot(list(closeness_deviation.keys()), list(closeness_deviation.values()), label='Closen
for node, value in closeness_deviation.items():
    plt.text(node, value, str(node), fontsize=8, ha='right')

plt.xlabel('Nodes')
plt.ylabel('Deviation from Average')
plt.title('Centrality Measure Deviations')
plt.legend()
plt.grid(True)
plt.show()

```

	Node	Betweenness Deviation	Degree Deviation	Closeness Deviation
0	1	0.533345	0.265000	0.200675
1	2	0.539988	0.348333	0.200675
2	3	-0.068406	-0.026667	0.000675
3	4	0.094638	0.056667	0.014468
4	5	-0.068406	-0.068333	-0.103029
5	6	-0.068406	-0.068333	-0.018373
6	7	-0.027645	0.056667	0.037039
7	8	-0.043647	0.015000	-0.018373
8	9	-0.068406	-0.026667	-0.070558
9	10	-0.008019	0.056667	0.037039
10	11	0.019457	0.056667	0.021728
11	12	-0.068406	-0.068333	-0.018373
12	13	0.054783	0.098333	0.045120
13	14	-0.068406	-0.068333	-0.018373
14	15	-0.068406	-0.068333	-0.103029
15	16	-0.068406	-0.068333	-0.018373
16	17	-0.068406	-0.026667	0.000675
17	18	-0.068406	-0.068333	-0.018373
18	19	-0.068406	-0.068333	-0.018373
19	20	-0.068406	-0.068333	-0.018373
20	21	-0.068406	-0.068333	-0.018373

21	22	-0.068406	-0.026667	0.007455
22	23	-0.068406	-0.068333	-0.099325
23	24	-0.068406	-0.026667	-0.005882
24	25	-0.068406	-0.068333	-0.018373

